Coursework Process Handbook(v1.1)

1. Getting Started and Forming Groups 3
   1.1. Lab Sessions During Week 1 3
   1.2. Forming Student Groups 3
   1.3. Deadline for Group Formation 3
   1.4. Creation of GitLab Groups 4
   1.5. Week 2 Checklist 4
2. Roles within the Group 4
   2.1. Organisational and Process Roles 4
   2.2. Development Roles 6
3. Team Interaction and Work Planning 6
   3.1. Kick-off Meeting 6
   3.2. Weekly Team Interaction 7
   3.3. Recommended Task Planning Process 7
   3.4. Project Plan and Gantt Chart 8
4. GitLab-Student 8
   4.1 Setting up your group GitLab project 8
   4.2 The GitLab Repository 9
      4.2.1 Structure 9
      4.2.2 Use Jupyter Notebooks 9
      4.2.3 The README 9
      Things you must add to your README for it to be deemed as a memo 10
      4.2.4 Formatting considerations 10
      4.2.5 Dependencies 11
   4.3 Data: The New Gold 11
      4.3.1 Storing your data in GitLab 11
      4.3.2 Keeping track of your data 11
      4.3.3 Providing data that cannot be uploaded or sourced publicly 12
   4.4 Submissions 12
5. Problems, Complaints and Resolution 12
Appendix A 13
   Instructions on using GitLab-Student 13
      Sign in to GitLab-Student 13
      Configure SSH Authentication 13
      Clone your repo and work on Jupyter 14

Introduction

This Handbook is intended for students and defines the process for organising work on the coursework project throughout the semester.

The requirements for the content, structure, outcomes, and assessment of the coursework are specified in a separate document. This Handbook does not duplicate those requirements. Instead, it describes the process of working on the coursework project: how teams are formed, how work is distributed, how team members interact, and how GitLab is used to support collaborative development.

The Handbook defines:

- the process for forming student teams;
- roles of team members and the distribution of responsibilities;
- principles of interaction between team members;
- the process for planning and allocating tasks;
- configuration management and organisation of work in GitLab;
- use of repositories, forks, commits, and merge requests;
- process for collaborative work on source code, documentation, and other project materials;
- organisation of laboratory sessions during which students work on their coursework projects;
- the process for presenting and discussing current progress;
- principles for recording completed tasks and the contribution of each team member;
- the process for interaction between the team and the lecturer during the project.

The purpose of this Handbook is to establish a clear and consistent process for all students working on coursework projects. Each team member should understand how collaborative work is organised, which tasks they are responsible for, how their work is integrated with the work of other team members, and how the team's progress is recorded.

Particular attention is given to teamwork and the use of GitLab as the main environment for organising the development process. The repository is used not only for storing source code, but also for coordinating work, managing changes, integrating the contributions of different team members, and tracking the development of the project.

A separate component of the process is Data Management — the rules for working with data and datasets used in the coursework project. This includes the process for selecting and obtaining datasets, checking the conditions of their use, complying with licensing restrictions, correctly acknowledging data sources, managing data storage and shared access within the team, and following any restrictions on the use, modification, and distribution of data. Students must also consider the legal and ethical aspects of data use and ensure that all datasets are used in accordance with their licences and with any conditions specified by the data owners or providers.

Lab sessions form part of this process. They are used for working on current project tasks, coordinating decisions within the team, demonstrating progress, and discussing problems that arise during development.

Students should use this Handbook as a practical guide to organising teamwork on the coursework project throughout the entire period of its development.

1. Getting Started and Forming Groups

Work on the coursework project begins during Week 1. During this week, the first lab sessions are held. These sessions introduce the organisation of the coursework project, present possible project topics, and explain the first steps in working with data and using GitLab.

1.1. Lab Sessions During Week 1

Since you are still trying to form teams, Week 1 is going to work a bit differently than the rest. We are going to invite you to attend the following sessions as follows:

Tuesday — Lab 1 (17:00): All Postgraduate (PG) students only;

Wednesday, 11:00-13:00 — Lab 2 & 3: All Undergraduate (UG) students only

Friday — Lab 4: no lab session will take place during Week 1.

If a student is unable to attend the recommended time slot, they may attend another lab session during Week 1 (apart from Lab 4).

During these sessions:

- the overall coursework project process will be explained;
- proposed project topics will be presented and discussed;
- the first steps in exploring data and datasets will be introduced;
- the basic principles of working with GitLab will be explained;
- the next steps required to begin team-based work will be outlined.

1.2. Forming Student Groups

During Week 1, students form their own teams in Canvas.

Students may form teams according to their own preferences. Each team should normally consist of 5 members (exceptions can be made on a case-by-case basis).

Groups are formed separately for: UG and PG students.

UG and PG students should not be combined within the same team.

Groups are formed through the People/Groups section of the course in Canvas.

Students may join an available group together with other students to form their team.

1.3. Deadline for Group Formation

Students may form their own groups until:

Monday of Week 2, 15:30.

By this time, each student should have joined a group in Canvas.

After this deadline, students who have not joined any group will be allocated to teams randomly. After this stage, the team composition will be treated as final for the subsequent coursework project process.

Every group will be assigned a lab session (there are 4 in total; every group may attend only one. Exceptions can be arranged on a case-by-case basis)

After the final group lists are published, changes to group membership are not permitted, except in exceptional cases that are reviewed and approved by the course leader individually.

1.4. Creation of GitLab Groups

During Week 2, you must fork the project template repository on GitLab and follow the instructions in Section 4.

Once all team members have been added in the group project, the GitLab group becomes the main environment for subsequent collaborative work on the coursework project, including dataset references, source code, documentation, and other project materials.

1.5. Week 2 Checklist

By the end of Week 2, each team should confirm that the following steps have been completed:

1. the team has been formed in Canvas;
2. the final list of team members has been confirmed;
3. the GitLab repository has been created and the team has been created added;
4. every team member has checked that they can access the GitLab group and project repository;
5. the team has reviewed the available coursework topics;
6. the team has made an initial decision about the topic or project direction;
7. the team has reviewed the available datasets relevant to the selected topic;
8. team members are ready to begin task allocation and project planning in the following stage.

2. Roles within the Group

The distribution of roles within the team is not strictly fixed. The team may organise its work differently depending on the selected topic, and the nature of the project.

However, a responsible team member, or several members, should be assigned to each major process activity and each technical development area. One student may perform several roles, and one role may be shared by several students if they are responsible for different tasks.

2.1. Organisational and Process Roles

Team Lead

The team should agree internally on who will perform the role of Team Lead.

The Team Lead:

1. performs the tasks on GitLab (Section 4)
2. coordinates the work of team members;
3. helps allocate tasks within the team;

4. monitors the overall progress of the project;
5. represents the team when communication with the lecturer is required;
6. coordinates maintenance of the main project `README`.

The Team Lead is not expected to perform all organisational work personally. The main purpose of this role is to coordinate the team.

Final Report Lead

The Final Report Lead coordinates the preparation of the final report and is responsible for its overall consistency and quality.

The responsibilities include:

1. organising the structure of the final report;
2. coordinating contributions from all team members;
3. checking consistency between different sections of the document;
4. checking formatting, references, tables, figures, and reported results;
5. carrying out the final review before submission.

Each team member is responsible for preparing the documentation related to the part of the project they have completed.

Configuration Management Lead

The Management Lead can be the Team Lead and is responsible for organising work in GitLab and maintaining the project repository in a consistent state.

The responsibilities may include:

1. managing team members' access to GitLab;
2. maintaining the repository structure and correspondence to the given template;
3. coordinating the merging of changes;
4. managing versions and tags;
5. helping resolve merge conflicts;
6. ensuring that important project changes are properly recorded in GitLab.

The Management Lead is not the only developer working with the repository. All team members are expected to use GitLab in accordance with the agreed development process.

Data Management Lead

The team should assign a person responsible for organising work with data.

The responsibilities include:

1. identifying and initially selecting relevant datasets;
2. checking the source of the data;
3. checking licences and conditions of us
4. coordinating the rules for storing and sharing data within the team.

Responsibility for Data Management may be shared between several students if the project uses several independent datasets.

Presentation and Pitch Lead

One or more team members may coordinate preparation of the final presentation and project pitch.

This role may include:

1. defining the structure of the presentation;
2. coordinating materials prepared by different team members;
3. selecting the key project results to present;
4. preparing demonstration materials;

5. coordinating the oral presentations.

This role does not mean that only one person should present the whole project. The presentation may be shared between several team members.

2.2. Development Roles

Every team member should participate in the technical work and contribute to the software implementation. It is recommended that development responsibilities are distributed across the following areas.

- EDA and Data Analysis
- Baseline Machine Learning Algorithms
- Neural Networks

A different distribution of development responsibilities may be used if required by the project. For example, two students may work on neural networks while taking responsibility for different architectures or different experiments.

3. Team Interaction and Work Planning

After the groups have been formed, students are required to attend the laboratory session allocated to their group, during which the first organisational team meeting — the kick-off group meeting — will take place.

3.1. Kick-off Meeting

The first meeting is facilitated by the Team Lead, who acts as the moderator. The Team Lead identifies the main issues that need to be discussed, facilitates the discussion, and helps the team reach agreed decisions.

During the meeting, team members should discuss, among other things:

- organisation of teamwork;
- initial allocation of roles;
- selection and clarification of the project topic;
- main project tasks;
- work with data;
- approaches to software implementation;
- planning of the next stages of work;
- upcoming milestones and deliverables.

It is recommended that the team records its main decisions and stores them in the GitLab repository. These may take the form of short meeting notes containing the date of the meeting, the issues discussed, the decisions made and the agreed next actions.

These records may later be used as material for the final report and as evidence of the team's ongoing activity throughout the project.

During each lab session, instructors and lecturers will normally allocate approximately 15–20 minutes to each team to discuss progress, plans, problems, and questions.

3.2. Weekly Team Interaction

Similar team meetings must be held every week during the allocated lab session.

During these meetings, the team should:

- review the results of the previous week;
- check the completion of planned tasks;
- discuss problems and possible delays;
- plan the next set of tasks;
- coordinate issues related to data and coding;
- discuss experimental results;
- coordinate the preparation of documentation;
- monitor preparation for milestones and deliverables.

The Team Lead facilitates the meeting, but all team members are expected to participate actively in the discussion and decision-making process.

3.3. Recommended Task Planning Process

The following approach is recommended for planning project work.

Stage 1. Task Identification

- The team discusses which tasks need to be completed during the next stage of the project.
- The discussion should result in a clear list of tasks.
- Simple collaborative techniques may be used to generate and organise ideas. For example, the team may use sticky notes, where each member proposes tasks and the team then discusses, groups, and structures them.

Stage 2. Task Evaluation

For each identified task, the team should discuss:

- estimated time required;
- level of difficulty;
- knowledge and skills required;
- dependencies on other tasks;
- required data, software tools, or other resources.

This helps the team establish a realistic order in which the work should be carried out.

Stage 3. Identification of Potential Assignees

For each task, it is recommended that the team identifies one or more potential assignees, considering:

- experience;
- technical skills;
- previous work on the relevant part of the project;
- team member’s interest in working on the task.

For some tasks, the team may identify both a primary developer and another team member responsible for review or testing.

Stage 4. Task Allocation

Once potential assignees have been identified, tasks should be allocated among team members with consideration of their current workload.

The allocation should be balanced and should avoid situations in which a substantial proportion of the work is repeatedly carried out by only one or two team members.

For each task, the following should be clear:

- responsible assignee;
- expected outcome;
- target completion date;
- dependencies on other tasks.

3.4. Project Plan and Gantt Chart

It is recommended that the team creates an overall project schedule showing:

1. main stages of the project;
2. key tasks;
3. milestones;
4. interim and final deliverables;
5. estimated completion dates;
6. dependencies between major tasks.

Any suitable tool for creating Gantt Charts may be used. The schedule should be updated regularly to reflect the team's actual progress.

An up-to-date Gantt Chart should preferably be included in the relevant team report or other deliverable, as it provides evidence of project planning and the practical organisation of teamwork.

4. GitLab-Student

For this coursework, you will be using GitLab as your collaborative platform. You will be able to discuss your project during the lab sessions each week.

If you have any questions or queries about the assessment, please do not hesitate to contact your nearest teaching team.

The repository for the project will be managed through **GitLab**. See CANVAS for more details and ask for help if you have any issues. This means that your project must be managed and maintained on GitLab (e.g., documentation, notebooks, scripts, etc).

4.1 Setting up your group GitLab project

1. **(All group members) Access to GitLab-Student:**
   Sign in to **https://gitlab-student.macs.hw.ac.uk/** (see how to do in **Appendix A**)
2. **Configuration Management Lead *f*orks the project template**
   a. Assuming you managed to sign in successfully to GitLab-Student, the project template should appear in the **list of projects:** **https://gitlab-student.macs.hw.ac.uk/f21dl-2026-27/f21dl-2026-27-students/f21dl-2026-27-template**
   b. The Configuration Management Lead **must click on the word *fork*** towards the top of the screen, then select their name when asked for a namespace. This will create your own copy of the project, which you can edit.

3. **Configuration Management Lead renames the repo**
   On Canvas you have been added to a numbered group (group1, group2, ...groupN). Please report the number of your group in the name of the forked repo. The name of the repo should appear like: "**f21dl-2026-27-PG-group-1**" or "**f21dl-2026-27-UG-group-1**". To do this, the Configuration Management Lead on their fork clicks "Settings" on the left, then "General", the beneath "Naming, description, topics" then change the name in the "Project name" box.
4. **Configuration Management Lead adds all the group members**
   The Configuration Management Lead **must manually add their student colleagues** in their group to their fork with the "**Maintainer**" role. To do this, the Configuration Management Lead on their fork clicks "Manage" then "Members" then clicks "Invite members", writes their team member's usernames in the box and click "Maintainer" in the "Role" drop down. Now all the group members can see and edit the same repo.

4.2 The GitLab Repository

Throughout the semester, **each group member is expected to individually commit their work and progress regularly to the repository**.[3] All the project work, including documentation, should be developed and managed on the repository. Each member is expected to contribute to the project to the deliverables of the course through regular commits, including informative comments/details/code reviews.[4] You will be evaluated with regards to having worked comparative hours on the project.

Importantly—and crucially—your entire project should be fully reproducible. Concretely, this means that anyone should be able to copy-paste commands to install dependencies and then run entire notebooks and nothing should crash, and everything should work entirely.

4.2.1 Structure

Throughout your project, ensure that your repository is **well-structured, organized, and follows best practices.**[5] This means that you should not need to guess where anything is: it should be explicit and everything should be clearly signposted. You should not have 10000 lines of code in a single file, or 100 lines of comments for a single function call *unless it is warranted.*

We've got you started with the following structure:

```
├─ data/          # Datasets
├─ notebooks/     # Jupyter notebooks for analysis/modelling
├─ scripts/       # Python scripts for preprocessing/ML
├─ documentation/  # Project docs and weekly updates
│  ├─ week1/
│  ├─ week2/
│  └─ ...
└─ README.md      # Project overview and data pipeline
```

The suggested structure should work for most people. However, this is *just a suggestion and if you have a better organisational structure, you should follow whatever is best for your project.*

Importantly, you should store files in well-named folders.

4.2.2 Use Jupyter Notebooks

For simplicity and ease, we strongly encourage you to use **Jupyter notebooks** to develop, evaluate, and log your tests/results.

4.2.3 The README

The README **should be treated as a memo:** a briefing note, more than a summary but not the full report, enough to get up to speed and refresh your memory without needing to spend hours refreshing your memory. Effective and succinct written communication is a critical skill that comes with practice (you're welcome).

Things you must add to your README for it to be deemed as a memo

1. Group Name
2. Project Title — A single-sentence title that succinctly describes your project
3. Group Members
4. Outline your Research objectives, as bullet points **[MSc Students only]**
5. *Your* Project Milestones, including when you are expecting to complete each requirement within context of the semester
6. The sources of your dataset(s), including:
    a. The *true* source of where the data is from[6]
    b. The original license for each source
    c. 2 specific examples from your datasets, presented nicely
    d. Any additional steps taken during data collection to expand the dataset, and any metrics associated with these. If you have not done this, don't worry.
7. All steps to install your project dependencies
8. How to run your data preparation pipeline, with a short overview of the steps it includes
9. A short description (100 words max.) of each requirements (R2–R4) and their location(s) within your repository using permanent links. This includes:
    a. What is your model predicting? (a.k.a., *the outputs*)
    b. What is your model using to predict from? (a.k.a., *the inputs*)
    c. A table of results (especially for R3/R4), using Markdown tables
    d. A figure showing the results, using permanent links

*Note: The **information you will present in your Project Pitch must also be on the GitLab README**. We have already provided the template you should follow. We expect you to complete up to (and including) the "Milestones".*

***[MSc Students]** You don't need to complete the Research objectives yet.*

4.2.4 Formatting considerations

Make use of all the markdown features that GitLab offers to present your work clearly and concisely. GitLab also have more advanced features too that do loads more, and it's all clearly outlined in their docs starting here.

GitLab supports Markdown. Jupyter Notebooks support Markdown. Use this opportunity to learn Markdown J

Here's some more considerations.

- You can use markdown headings as liberally as you would like to clearly demonstrate each section.
- Use code blocks when including code
- Directly link to your Jupyter notebooks using permanent links, so they can be found
- Use collapsed sections when you would like to keep something interesting or notable, but might be outside the scope of the checklist.
- Use markdown tables to illustrate simple information where relevant
- Use graphs where possible, but tables when you can't use graphs
- The GitLab/GitHub markdown file is just a cool HTML page, so you can include interactive images and videos and audio if that can further help

- Do not use huge graphs if they are not needed. Images and pieces can be smaller to take up less space on the screen.

4.2.5 Dependencies

For your projects, there should be a single requirements.txt or environment.yml (or similar) that explicitly details the dependencies and the versions used in the project. There should be only one file, and it should be stored in the root of the project.

Also, within your dependencies list, you must only mention each package used once, and you must mention the explicit version of the package used. If different members of your group used different versions of packages, you must find a common version and verify that all aspects of your project work with that package version.

4.3 Data: The New Gold

Managing data is a very real problem in the world of machine learning and is something that can only be learned through experience.

4.3.1 Storing your data in GitLab

GitLab storage capacity is limited. The maximum file attachment limit is 2MB, the maximum push size (of commits) limit is 5MB. You can verify the storage quota of your repo: in the left sidebar, select Settings > Usage quotas.

As a result, we recommend the following approach for handling your raw data.

As you are using publicly available datasets, we highly recommend that you **create a clear and consistent pipeline that will automatically download, extract, organise, and pre-process your dataset.** This automated pipeline will ensure that everyone in your group uses an identical copy of the dataset.[7] This script/pipeline should be created and communicated as early as possible, before any modelling or EDA is undertaken.

If you need to change this process, you should communicate to all your group members that they need to re-run the script. We also recommend creating a dedicated section within the README for documenting the dates and reasoning for why your pipeline was changed.

Raw data with a size no larger than 50MiB can be stored in your GitLab repo (if you want). In all the other cases, we recommend using external storage resources (e.g., OneDrive). The link of the externally stored data should be reported in the README. The process for extracting this data should be part of the automated pipeline.

Use of external resources to store your data

When you use external resources to store the data (e.g., OneDrive), you can use a command like wget in your script to download it. It's important to remember that you can include the file's name in the .gitignore file in the project, so the `git add` command will ignore that file.

Splitting the dataset in multiple files

If the size of the dataset is very large, we recommend compressing and splitting your archives into multiple files. The process of the use of these files should be part of your pipeline.

4.3.2 Keeping track of your data

It is extremely important that you keep a copy of your raw data separate to your pre-processed data. This is to aid with reproducibility for any other researchers. Your raw data should (ideally) get committed into the repository once you have decided on your datasets—unless your dataset is too large for GitLab—and then not touched. If you need to change the raw data for

any reason, you should clearly document your reasons within a commit and/or a pull request so that it can be tracked.

4.3.3 Providing data that cannot be uploaded or sourced publicly

If for some reason—of which there are many valid ones—you have curated data that cannot be uploaded to the repository or sourced publicly, one person from your group should upload it to their university OneDrive account and enable sharing with "People in Heriot-Watt University". If you do not enable sharing, no one can access your data, which is the same as not providing it at all.

4.4 Submissions

You must not make any new commits to your repository after the deadline. Groups that make commits after the deadline should consult the coursework specification for details on late submissions.

5. Problems, Complaints and Resolution

During the project, various issues or difficulties may arise and should be addressed promptly. If a student is absent from the session where the group distributes tasks, the student is responsible for any missed agreements and must contact the group independently to clarify their responsibilities.

Each group member is responsible for:

- participating in the group's work;
- completing agreed-upon tasks by the established deadlines;
- maintaining communication with other group members;
- informing the group and the course leader about any issues that prevent task completion;
- attending lab sessions dedicated to group work.

If a student does not complete the agreed-upon tasks, does not participate in the group's work, or ignores communication with the group, their individual grade may be reduced or not assigned

Problems in group work must be addressed in a timely manner, not after the project has been completed. If a group member does not complete their tasks, ignores messages, does not participate in the work, or creates other difficulties for the group, the other group members must inform the course leader during a class session dedicated to group work or before the established intermediate deadline.

A complaint must be specific and include a description of the problem. For example:

- what task was assigned to the student;
- what deadline was agreed upon;
- what exactly was not completed;
- what communication attempts were made;
- how this affected the group's work.

General statements such as "he/she does not do anything" without specific examples may be insufficient for reviewing an individual grade.

Complaints submitted only after the project has been completed may not be considered if the problem was not reported earlier and the group had an opportunity to contact the course leader during the project.

Lab sessions dedicated to group work are the primary time for organising work, distributing tasks, discussing problems, and resolving conflicts. If a student does not attend such sessions, the student may not claim that they were not informed about group decisions, task distribution, or internal deadlines. Complaints from students who were not present at the sessions where the relevant issue was discussed may be considered invalid, except in cases of documented and justified absence.

Appendix A

Instructions on using GitLab-Student

GitLab-Student is a code repository (i.e. somewhere on the internet where code in stored) that is hosted by the University. Lecturers use it to share code with students, and students use it to store code that they have written for assignments. This introduction will take you through the basics.

Sign in to GitLab-Student

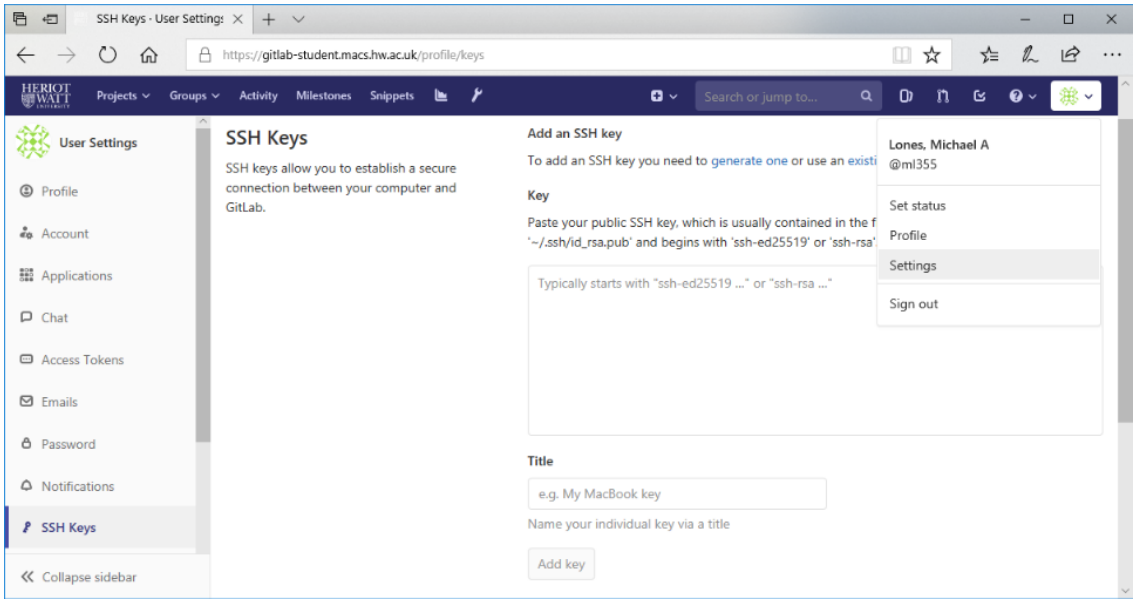
Before you can use GitLab-Student, you need to sign in:

1. Open a web browser and go to: https://gitlab-student.macs.hw.ac.uk/
2. This will bring up the sign in screen. Select the “Remember me” checkbox and then click on Azure OAuth2.
3. Type in your university email address (your user name followed by @hw.ac.uk) and click **Next**.
4. This should take you to the university’s sign in page. Enter your email address and password and click **Sign In**.

Configure SSH Authentication

GitLab uses public key encryption to securely transfer code. Follow the instruction on https://docs.gitlab.com/user/ssh/

1. Generate the SSH key.
2. Now go back to GitLab-Student in your web browser, click on the right-most dropdown menu in GitLab-Student’s top toolbar and choose **Settings**, and click on the **SSH keys** option on the left.
3. Paste the public key and click **Add key**. Now you can verify the SSH connection.



Clone your repo and work on Jupyter

1. Clone your GitLab repo to your local computer, following the command on https://docs.gitlab.com/topics/git/clone/
2. Navigate to the local repository from Jupyter and start coding!